ĐẶC TẢ KỊCH BẢN FIRMWARE

Hệ thống giám sát mực nước và cảnh báo lũ quét

Thiết bị: Bringter Flood Monitor · MCU: nRF52840 (Zephyr RTOS) · Modem: BC660K-GL NB-IoT

Phiên bản: 1.0 — Tháng 07/2025	Đối tượng: Sinh viên / Đội firmware	Trạng thái: Bản nháp nội bộ

Tài liệu này mô tả đầy đủ và chi tiết các kịch bản vận hành mà firmware của thiết bị Bringter Flood Monitor cần xử lý. Tất cả các biến số, ngưỡng và cơ chế đều được giải thích từ đầu để sinh viên mới tham gia dự án có thể hiểu và lập trình theo đúng yêu cầu nghiệp vụ, không cần tra cứu thêm tài liệu khác.

I. Tổng quan hệ thống và phần cứng

1. Mục đích thiết bị

Bringter Flood Monitor là thiết bị IoT đặt ngoài trời, lắp trên cầu, kênh, suối hoặc hố ga, dùng để đo mực nước định kỳ và gửi dữ liệu lên hệ thống ThingsBoard qua mạng NB-IoT. Khi mực nước hoặc tốc độ dâng nước vượt ngưỡng nguy hiểm, thiết bị tự động chuyển sang chế độ cảnh báo, tăng tần suất đo và gửi dữ liệu liên tục để hỗ trợ điều phối ứng cứu lũ quét.

Thiết bị chạy bằng pin, không có điện lưới — do đó tiết kiệm năng lượng là yêu cầu thiết kế quan trọng nhất, ảnh hưởng trực tiếp đến mọi quyết định kiến trúc firmware.

2. Sơ đồ phần cứng liên quan đến firmware

Sinh viên cần nắm các thành phần sau để hiểu tại sao firmware hoạt động theo cách mô tả trong tài liệu này:

Thành phần	Chi tiết	Vai trò trong firmware
MCU	`nRF52840 (ARM Cortex-M4)`	Chạy toàn bộ firmware Zephyr RTOS. Có thể ngủ sâu (System OFF) ở mức ~2 µA. Có RTC nội bộ giữ thời gian khi ngủ.
Cảm biến đo mực nước	`JSN-SR04T (siêu âm chống nước)`	Phát xung siêu âm, đo thời gian echo để tính khoảng cách từ cảm biến xuống mặt nước. Tầm đo 20–600 cm.
Modem truyền dữ liệu	`BC660K-GL (NB-IoT LTE Cat-NB2)`	Kết nối mạng di động để gửi dữ liệu lên ThingsBoard qua MQTT. Tiêu thụ ~100–300 mA khi gửi — thành phần ngốn pin nhất.
Cảm biến nhiệt độ board	`STTS22H (I2C, gắn trên PCB)`	Đo nhiệt độ bên trong hộp thiết bị. Dùng để giám sát sức khỏe phần cứng, KHÔNG phải nhiệt độ ngoài trời.
Nguồn điện	`Pin LiPo tích hợp (14.5 Ah)`	Cấp toàn bộ điện. Điện áp 3.0–4.2 V. Firmware đọc ADC để biết % pin còn lại.
Lưu trữ cấu hình	`EEPROM ngoài`	Lưu các tham số cấu hình thiết bị (vị trí lắp, ngưỡng cảnh báo...) — giữ nguyên khi tắt nguồn, khác với RAM.

3. Cách thiết bị đo mực nước

Cảm biến JSN-SR04T lắp phía trên, hướng xuống dưới. Nó phát ra xung siêu âm và đo thời gian phản xạ về — từ đó tính được khoảng cách từ cảm biến đến mặt nước (gọi là water_distance_cm).

Mực nước thực tế tính theo công thức:

```
water_level_cm = install_height_cm - water_distance_cm

  install_height_cm  : chiều cao lắp cảm biến so với đáy kênh/sông (cm)
                       → lưu trong EEPROM khi lắp đặt thực địa
  water_distance_cm  : khoảng cách đo được từ cảm biến xuống mặt nước (cm)
                       → -1 nếu cảm biến lỗi hoặc timeout
  water_level_cm     : mực nước tính từ đáy (cm)
                       → âm khi chưa có nước (bình thường)
                       → dương khi có nước
```

Ví dụ: Cảm biến lắp ở độ cao 200 cm so với đáy sông. Cảm biến đo được khoảng cách xuống mặt nước là 145 cm. → Mực nước = 200 − 145 = **55 cm**.

Ngoài giá trị tuyệt đối, firmware còn tính thêm phần trăm đầy theo công thức:

```
water_level_pct = (water_level_cm / install_height_cm) × 100

  → 0%   khi sông khô hoàn toàn
  → 100% khi nước đã dâng đến đúng vị trí lắp cảm biến
  → >100% khi nước dâng vượt qua vị trí lắp cảm biến (ngập nghiêm trọng)
           Backend KHÔNG được cap giá trị này — để nguyên số thực.
```

4. Các tham số cấu hình đọc từ EEPROM

Khi thiết bị khởi động, firmware đọc các tham số sau từ EEPROM. Những giá trị này do kỹ thuật viên ghi vào lúc lắp đặt thực địa, khác nhau giữa các điểm triển khai:

Tham số	Ví dụ	Ý nghĩa
install_type	lu_quet	Loại vị trí lắp đặt. Quyết định toàn bộ bộ tham số vận hành (xem bảng ở mục II).
install_height_cm	200	Chiều cao lắp cảm biến so với đáy kênh/sông (cm). Dùng để tính water_level_cm.
location_name	Thuong nguon suoi C	Tên vị trí. Hiển thị trên dashboard, không ảnh hưởng logic firmware.
threshold_pct	40	Ngưỡng % mực nước để kích hoạt chế độ khẩn cấp. Tự động set theo install_type nếu không override.
threshold_rate	5.0	Ngưỡng tốc độ dâng (cm/phút) để kích hoạt khẩn cấp. Tự động set theo install_type.
interval_normal_s	300	Chu kỳ đo ở chế độ bình thường (giây). Ví dụ: 300 = 5 phút.
interval_emergency_s	30	Chu kỳ đo ở chế độ khẩn cấp (giây). Ví dụ: 30 = 30 giây.

mqtt_access_token	abc123xyz	Mã xác thực để kết nối ThingsBoard. Duy nhất cho mỗi thiết bị.
device_id	FLOOD_HN_001	ID thiết bị. Dùng trong MQTT Client ID: floodmon_FLOOD_HN_001.

5. Phân biệt install_type — tại sao cần?

Đặc tính thay đổi mực nước rất khác nhau giữa lũ quét vùng núi, sông lớn và hố ga đô thị. Nếu dùng cùng một bộ tham số cho tất cả sẽ hoặc lãng phí pin (đo quá dày ở sông lớn) hoặc bỏ lỡ cảnh báo (đo quá thưa ở lũ quét). install_type giải quyết điều này bằng cách tự động chọn bộ tham số phù hợp:

install_type	interval_normal	interval_emergency	threshold_pct	threshold_rate	Đặc điểm
lu_quet	**5 phút**	**30 giây**	40%	5 cm/phút	Lũ quét núi — cực nhanh, nguy hiểm nhất. Phản ứng dưới 1 phút.
song / kenh	**15 phút**	**2 phút**	60%	2 cm/phút	Sông lớn — dâng chậm (nhiều giờ). Ưu tiên tiết kiệm pin.
ho_ga / cot_den	**5 phút**	**1 phút**	50%	3 cm/phút	Ngập đô thị — dâng 15–30 phút theo mưa.

II. Mô hình trạng thái (State Machine) và cơ chế độ trễ

Firmware không chạy theo vòng lặp cố định. Thay vào đó, thiết bị vận hành theo mô hình trạng thái (State Machine) gồm 2 chế độ: BÌNH THƯỜNG và KHẨN CẤP. Mỗi chế độ có chu kỳ đo, cách xử lý mạng và hành vi gửi dữ liệu hoàn toàn khác nhau.

Thiết bị tự động chuyển giữa hai chế độ dựa trên giá trị đo được — không cần lệnh từ server. Để tránh hiện tượng "nhấp nháy" (Ping-pong) khi mực nước dao động liên tục quanh ngưỡng, hệ thống áp dụng cơ chế Hysteresis — yêu cầu xác nhận nhiều chu kỳ liên tiếp trước khi hạ cấp trạng thái.

1. Trạng thái BÌNH THƯỜNG (Normal State)

1.1. Điều kiện để ở trạng thái BÌNH THƯỜNG

Thiết bị ở chế độ BÌNH THƯỜNG khi thoả đồng thời cả hai điều kiện sau:

- **Mức nước:** water_level_pct (%) **nhỏ hơn** threshold_pct (ngưỡng % đọc từ EEPROM theo install_type).
- **Tốc độ dâng:** rate_of_rise_cm_per_min (cm/phút) **nhỏ hơn** threshold_rate (ngưỡng tốc độ đọc từ EEPROM).

Nếu đang hạ từ KHẨN CẤP xuống, cần thêm điều kiện xác nhận 2 chu kỳ liên tiếp — xem chi tiết ở mục **3. Hysteresis**.

1.2. Firmware làm gì ở chế độ BÌNH THƯỜNG

- **Nghiệp vụ đo đạc:** MCU thức dậy từ trạng thái ngủ sâu (System OFF) sau mỗi `interval_normal_s` giây. Thực hiện đo mực nước mất khoảng ~100ms, rồi ngủ lại ngay.
- **Nghiệp vụ mạng: Modem BC660K tắt hoàn toàn** — không kết nối, không gửi dữ liệu. Đây là lý do chính giúp tiết kiệm pin: modem là thành phần tiêu thụ điện lớn nhất (~3 mA idle, ~300 mA khi gửi).
- **Lưu trữ dữ liệu:** Mỗi lần đo xong, firmware lưu giá trị vào Ring Buffer trong RAM. Ring Buffer chứa tối đa 48 bản ghi. Dữ liệu trong RAM được giữ nguyên qua các chu kỳ ngủ sâu (nRF52840 hỗ trợ điều này).
- **Kiểm tra ngưỡng:** Sau mỗi lần đo, firmware so sánh water_level_pct và rate_of_rise với threshold. Nếu vượt ngưỡng → chuyển sang KHẨN CẤP ngay trong chu kỳ đó, không chờ chu kỳ tiếp theo.

> ✓**Nguyên tắc tiết kiệm pin:** Ở BÌNH THƯỜNG, mỗi chu kỳ 5 phút, MCU chỉ thức ~100ms để đo. Phần còn lại của 5 phút (= 299.9 giây) toàn bộ thiết bị ngủ ở ~2–15 µA. Đây là cơ chế chính giúp pin sống 4–14 tháng.

2. Trạng thái KHẨN CẤP (Emergency State)

2.1. Điều kiện chuyển sang KHẨN CẤP

Thiết bị chuyển sang KHẨN CẤP khi thoả ÍT NHẤT MỘT trong hai điều kiện (không cần cả hai):

- **Điều kiện 1 — Mức nước:** water_level_pct ≥ threshold_pct.
- **Điều kiện 2 — Tốc độ dâng:** rate_of_rise_cm_per_min ≥ threshold_rate.

Tại sao dùng OR (hoặc)? Lũ quét đặc biệt nguy hiểm vì tốc độ dâng rất nhanh ngay từ đầu, nhưng mực nước tuyệt đối có thể vẫn còn thấp. Nếu chỉ dùng điều kiện mức nước, sẽ bỏ lỡ cảnh báo giai đoạn đầu. Ví dụ: mực nước mới ở 25% nhưng tốc độ dâng đã là 8 cm/phút → firmware phát hiện ngay qua Điều kiện 2.

> **Lưu ý đặc biệt với lu_quet:** threshold_pct = 40% — thấp hơn các loại khác. Lý do: lũ quét ở vùng núi có thể dâng 1 mét chỉ trong 30 phút. Nếu chờ đến 50–60% sẽ mất cơ hội cảnh báo sơ tán. Khi đạt 40% tốc độ dâng thường đã là 10–15 cm/phút.

2.2. Firmware làm gì ở chế độ KHẨN CẤP

- **Bật Modem ngay lập tức:** Ngay khi phát hiện ngưỡng bị vượt, firmware bật BC660K và thiết lập kết nối MQTT đến ThingsBoard.
- **Giữ kết nối MQTT liên tục:** Khác với BÌNH THƯỜNG (connect → gửi → disconnect), ở KHẨN CẤP thiết bị giữ kết nối liên tục trong suốt sự kiện. Lý do: mỗi lần kết nối lại mất 12–20 giây handshake NB-IoT — quá lâu khi cần gửi mỗi 30 giây.
- **Đo và gửi liên tục:** Cứ mỗi `interval_emergency_s` giây, đo xong là gửi ngay lên ThingsBoard — không qua Ring Buffer. Gói tin gửi kèm `"report_type": "event"` để backend ưu tiên xử lý.
- **Tiếp tục kiểm tra ngưỡng:** Sau mỗi lần đo, firmware kiểm tra xem đã đủ điều kiện hạ về BÌNH THƯỜNG chưa (xem Hysteresis).

3. Cơ chế Hysteresis — tránh Ping-pong

Giả sử mực nước dao động quanh ngưỡng 40%: lúc 39%, lúc 41%, lúc 38%... Nếu firmware chuyển trạng thái ngay sau mỗi lần đo, thiết bị sẽ liên tục bật/tắt modem — lãng phí pin và gây nhiễu cảnh báo. Cơ chế Hysteresis ngăn điều này.

3.1. Quy tắc hạ từ KHẨN CẤP → BÌNH THƯỜNG

Firmware KHÔNG hạ cấp ngay khi điều kiện KHẨN CẤP không còn thoả mãn. Thay vào đó:

- **Điều kiện để bắt đầu đếm:** Cả hai giá trị đều dưới ngưỡng: water_level_pct < threshold_pct VÀ rate_of_rise < threshold_rate.
- **Chu kỳ xác nhận 1:** Điều kiện thoả → biến `confirm_count` tăng lên 1.
- **Chu kỳ xác nhận 2:** Điều kiện vẫn thoả ở chu kỳ kế tiếp → confirm_count tăng lên 2 → chuyển về BÌNH THƯỜNG, tắt Modem.
- **Nếu vượt ngưỡng trở lại:** Trong bất kỳ chu kỳ nào mà một trong hai điều kiện KHẨN CẤP thoả lại → `confirm_count = 0` → tiếp tục KHẨN CẤP.

```
// Chạy sau mỗi lần đo, khi đang ở EMERGENCY
void check_if_can_downgrade(uint8_t pct, float rate) {
    bool both_below = (pct < threshold_pct) && (rate < threshold_rate);
    if (both_below) {
        confirm_count++;
        if (confirm_count >= 2) {
            confirm_count = 0;
            switch_to_normal();   // Hạ cấp, tắt modem
        }
    } else {
        confirm_count = 0;        // Reset nếu lại vượt ngưỡng
    }
}
```

Giá trị nào được ghi nhận? Giá trị của chu kỳ đo xác nhận cuối cùng (lúc confirm_count = 2) là giá trị chính thức được ghi vào Ring Buffer và gửi lên ThingsBoard — không dùng giá trị của chu kỳ vượt ngưỡng trước đó.

III. Xử lý dữ liệu — lọc nhiễu và lỗi phần cứng

Cảm biến siêu âm JSN-SR04T hoạt động tốt trong điều kiện bình thường, nhưng trong môi trường lũ quét có nhiều yếu tố gây nhiễu: mưa lớn đập thẳng vào cảm biến, bọt nước trên mặt sông, vật nổi (rác, cành cây), côn trùng. Firmware cần phân biệt nhiễu thoáng qua với tín hiệu thật, đồng thời xử lý trường hợp cảm biến bị lỗi phần cứng thực sự.

1. Bộ lọc nhiễu bất đối xứng (Asymmetric Noise Filter)

Gọi là "bất đối xứng" vì cách xử lý hoàn toàn khác nhau giữa trường hợp mực nước TĂNG và GIẢM đột biến:

1.1. Khi mực nước TĂNG ĐỘT BIẾN (delta > 30% trong 1 chu kỳ)

Ví dụ: chu kỳ trước đo được 25%, chu kỳ này đọc 60% — nhảy 35% chỉ trong 5 phút. Nguyên nhân có thể là lũ thật, hoặc có thể là vật nổi chắn cảm biến. Firmware xử lý như sau:

- **Bước 1 — Chưa kết luận ngay:** Không chuyển trạng thái, không ghi vào Ring Buffer.
- **Bước 2 — Ngủ ngắn 5 giây:** Firmware dùng k_sleep(K_SECONDS(5)) — đủ để vật nổi trôi qua, không đủ để lũ thật rút đi.
- **Bước 3 — Đo lại (Double-check):** Đo thêm lần nữa để xác nhận.

Kết quả Double-check dẫn đến một trong hai trường hợp:

- **Trường hợp A — Xác nhận tăng thật:** Lần đo lại vẫn cho giá trị cao (chênh ít hơn 5% so với lần 1). → Ghi nhận giá trị của **lần đo lại** làm giá trị chính thức cho chu kỳ này. → Kích hoạt chuyển KHẨN CẤP nếu vượt ngưỡng.
- **Trường hợp B — Xác nhận là nhiễu:** Lần đo lại trả về giá trị thấp (gần với lần trước). → Loại bỏ hoàn toàn lần 1. → Ghi nhận giá trị của **lần đo lại** làm giá trị chính thức. → Không chuyển trạng thái.

Lưu ý về tăng chậm: Nếu mực nước tăng từ từ (delta ≤ 30% trong 1 chu kỳ), ghi nhận bình thường, không cần Double-check.

```
MeasureResult handle_reading(uint8_t prev_pct, uint8_t new_pct) {
    int delta = (int)new_pct - (int)prev_pct;
    if (delta > 30) {
        k_sleep(K_SECONDS(5));           // Chờ nhiễu qua
        uint8_t confirm = measure();     // Đo lại
        // Lần đo lại luôn là giá trị chính thức — dù kết quả nào
        if (confirm >= new_pct - 5) {
            return CONFIRMED(confirm);   // Tăng thật, kích hoạt cảnh báo
        } else {
            return NOISE(confirm);       // Nhiễu, không cảnh báo
```

```
        }
    }
    return NORMAL(new_pct);              // Tăng chậm, ghi bình thường
}
```

1.2. Khi mực nước GIẢM (hạ cấp trạng thái)

Khi đang ở KHẨN CẤP và phát hiện mực nước đã giảm, firmware áp dụng quy tắc Hysteresis đã mô tả ở mục II.3 — yêu cầu 2 chu kỳ liên tiếp xác nhận. Điều này phòng ngừa trường hợp sóng cao thoáng qua làm mực nước đọc thấp hơn thực tế trong một chu kỳ.

> ⚠**Nguyên tắc nhất quán dữ liệu:** Double-check tức thì (ngủ 5 giây) KHÔNG sinh ra thêm bản ghi trong Ring Buffer. Mỗi chu kỳ đo (interval_normal_s hoặc interval_emergency_s) chỉ chốt đúng 1 giá trị hợp lệ duy nhất. Quy tắc này đảm bảo thuật toán Backwards Timestamping (phần V) tính đúng — không bị sai lệch nhịp.

2. Xử lý lỗi phần cứng cảm biến (Sensor Fault)

Khác với nhiễu thoáng qua, lỗi phần cứng là khi cảm biến không hoạt động được: I2C mất kết nối, hoặc cảm biến liên tục trả về -1 (timeout). Đây là tình huống nghiêm trọng hơn, cần xử lý khác:

- **Ngưỡng kết luận lỗi:** Nếu water_distance_cm = -1 liên tiếp ≥ 3 chu kỳ đo → kết luận phần cứng lỗi (không chỉ là nhiễu).
- **Đóng băng trạng thái:** Firmware KHÔNG tự suy luận "sông đang khô" hay "lũ đã rút" chỉ vì không đọc được cảm biến. Giữ nguyên trạng thái và giá trị mực nước hợp lệ cuối cùng.
- **Bắt buộc báo cáo ngay:** Bật Modem (ngay cả khi đang ở BÌNH THƯỜNG), gửi gói tin lên ThingsBoard với trường `sensor_fault: true` để yêu cầu kiểm tra thực địa.
- **Trên dashboard:** Giữ hiển thị giá trị mực nước hợp lệ cuối cùng — không hiển thị 0 hay để trống.

```
static int fault_count = 0;

void handle_sensor_value(int distance_cm) {
    if (distance_cm == -1) {
        fault_count++;
        if (fault_count >= 3) {
            // Phần cứng lỗi thật sự
            freeze_current_state();      // Không tự chuyển trạng thái
            force_modem_on();
            publish_sensor_fault();      // Báo cáo ngay lên ThingsBoard
            fault_count = 0;
        }
```

```
        // Giữ nguyên last_valid_level_cm — không cập nhật
        return;
    }
    fault_count = 0;                    // Reset nếu cảm biến ổn lại
    update_water_level(distance_cm);
}
```

IV. Quản lý bộ nhớ và Ring Buffer

Ở chế độ BÌNH THƯỜNG, thiết bị không gửi dữ liệu ngay. Thay vào đó, mỗi lần đo xong, giá trị được lưu vào một vùng nhớ đặc biệt trong RAM gọi là Ring Buffer. Khi có kết nối (vào KHẨN CẤP hoặc sau sự kiện), toàn bộ dữ liệu được gửi lên ThingsBoard một lần.

1. Ring Buffer là gì?

Ring Buffer (bộ đệm vòng) là một mảng có kích thước cố định. Khi đầy, thay vì từ chối ghi thêm, nó ghi đè lên dữ liệu cũ nhất — như một cái vòng tròn xoay liên tục.

- **Dung lượng:** Tối đa 48 bản ghi. Với chu kỳ BÌNH THƯỜNG 30 phút, 48 bản ghi = 24 giờ dữ liệu. Với chu kỳ 5 phút = 4 giờ dữ liệu.
- **Lưu qua Deep Sleep:** RAM của nRF52840 duy trì nội dung qua các chu kỳ ngủ sâu (System OFF). Dữ liệu không mất khi MCU ngủ — chỉ mất khi tháo pin hoàn toàn.
- **FIFO Overwrite:** Nếu mạng NB-IoT mất kết nối kéo dài và Ring Buffer đã đầy 48 bản ghi, firmware tự động ghi đè bản ghi cũ nhất. Đây là quyết định nghiệp vụ đã được chấp thuận — ưu tiên không bị tràn RAM (crash thiết bị) hơn là giữ lại dữ liệu cũ.

> ⚠ **Retry khi gửi thất bại:** Khi bật Modem để gửi dữ liệu, firmware chỉ thử tối đa 3 lần. Nếu cả 3 lần thất bại → tắt Modem ngay, dữ liệu vẫn ở Ring Buffer, chờ chu kỳ sau thử lại. KHÔNG được thử vô hạn lần — sẽ cạn pin.

2. Xóa Ring Buffer — chỉ khi có ACK

Firmware chỉ xóa Ring Buffer (reset về rỗng) khi nhận được xác nhận thành công từ ThingsBoard: chuỗi `+QMTPUB: 0,0,0` trả về từ modem sau lệnh gửi. Nếu gửi nhưng không nhận được ACK (timeout, mất sóng giữa chừng) → giữ nguyên dữ liệu trong Ring Buffer để thử gửi lại lần sau.

V. Đồng bộ thời gian (Time Synchronization)

ThingsBoard cần biết dữ liệu đo được vào lúc mấy giờ để vẽ đúng đồ thị theo thời gian. Thiết bị không có GPS, không kết nối internet liên tục — nên không thể lấy giờ chuẩn theo cách thông thường. Firmware giải quyết bài toán này bằng kỹ thuật Backwards Timestamping.

1. Backwards Timestamping — tính ngược thời gian

Mỗi khi bật Modem (vào KHẨN CẤP, hoặc sau sự kiện để gửi Ring Buffer), firmware thực hiện:

- **Bước 1 — Lấy giờ mạng:** Gửi lệnh AT `AT+CCLK?` đến modem BC660K. Modem trả về giờ hiện tại lấy từ mạng NB-IoT của nhà mạng (Viettel, Mobifone...).
- **Bước 2 — Đồng bộ RTC nội bộ:** Cập nhật bộ đếm RTC của nRF52840 theo giờ mạng vừa lấy. RC Oscillator của nRF52840 có thể sai số (drift) vài giây mỗi ngày khi ngủ sâu — việc đồng bộ này khắc phục drift tích lũy.
- **Bước 3 — Tính ngược timestamp:** Biết giờ hiện tại và biết mỗi bản ghi cách nhau đúng interval_normal_s giây → tính ngược Unix Timestamp cho từng bản ghi trong Ring Buffer.

```
void sync_and_backfill_timestamps() {
    // Lấy giờ từ mạng
    uint64_t now_ms = modem_get_network_time();  // AT+CCLK?

    // Đồng bộ RTC nội bộ
    rtc_set(now_ms);

    // Tính ngược timestamp cho từng bản ghi
    // Bản ghi mới nhất (index = count-1) có timestamp = now_ms
    // Bản ghi trước đó cách now_ms đúng 1 chu kỳ
    int n = ring_buffer.count;
    for (int i = 0; i < n; i++) {
        // i=0 là bản ghi cũ nhất, i=n-1 là mới nhất
        uint64_t offset_ms = (uint64_t)(n - 1 - i) * interval_normal_s *
1000;
        ring_buffer.records[i].ts = now_ms - offset_ms;
    }
}
```

> ⚠ **Backend phải dùng device timestamp:** ThingsBoard phải được cấu hình dùng trường 'ts' từ payload thiết bị, KHÔNG dùng server-side timestamp. Lý do: khi gửi Ring Buffer, các gói tin mang ts trong quá khứ. Nếu ThingsBoard tự gán thời gian nhận được, toàn bộ chuỗi lịch sử sẽ sai — mọi bản ghi đều hiển thị vào cùng một thời điểm.

VI. Quản lý kết nối NB-IoT và cơ chế retry

Mạng NB-IoT ở vùng núi, vùng sâu không ổn định. Firmware cần xử lý đúng khi không kết nối được — vừa đảm bảo dữ liệu không mất, vừa không lãng phí pin vào việc thử kết nối vô hạn.

1. Giao thức kết nối

- **Giao thức truyền tải:** UDP (User Datagram Protocol) — không cần bắt tay hai chiều, phù hợp với NB-IoT tiết kiệm pin.
- **Giao thức ứng dụng:** MQTT v3.1.1 — gửi telemetry đến ThingsBoard qua topic `v1/devices/me/telemetry`.
- **QoS 0 — Fire and Forget:** Firmware gửi gói tin và không chờ xác nhận từng gói (ngoại trừ ACK cuối để xóa Ring Buffer). Chấp nhận mất gói đổi lấy tiết kiệm pin.

2. Quy trình retry khi gửi thất bại

- **Tối đa 3 lần thử:** Mỗi lần gửi (dù BÌNH THƯỜNG hay KHẨN CẤP), firmware thử tối đa 3 lần. Giữa mỗi lần retry chờ 30 giây.
- **Sau 3 lần thất bại → tắt modem ngay:** Không thử thêm. Tắt Modem để bảo vệ pin. Dữ liệu vẫn trong Ring Buffer.
- **Ngủ 3–6 giờ:** Sau khi tắt modem vì thất bại liên tiếp, thiết bị ngủ 3–6 giờ trước khi thử lại — tránh cạn pin vì dò mạng liên tục ở vùng sóng yếu.
- **Offline timeout 48 giờ:** Nếu không kết nối được trong 48 giờ liên tục, thiết bị đánh dấu trạng thái offline. Dashboard ThingsBoard sẽ hiển thị thiết bị mất kết nối.

```
#define MAX_RETRY            3
#define RETRY_INTERVAL_S    30       // giây giữa mỗi retry
#define SLEEP_AFTER_FAIL_S  (3*3600) // 3 giờ
#define OFFLINE_TIMEOUT_S  (48*3600) // 48 giờ

bool send_with_retry(Payload *p) {
    for (int i = 0; i < MAX_RETRY; i++) {
        if (mqtt_publish(p) == SUCCESS) {
            last_success_ts = rtc_now();
            return true;
        }
        k_sleep(K_SECONDS(RETRY_INTERVAL_S));
    }
    // Thất bại — tắt modem, ngủ dài
    modem_power_off();
    schedule_wakeup_s(SLEEP_AFTER_FAIL_S);
    sys_poweroff();
    return false;
}
```

VII. Cập nhật firmware từ xa (OTA / FOTA)

Sau khi triển khai hàng chục hoặc hàng trăm thiết bị ngoài thực địa (vùng núi, trên cầu, hố ga...), việc đến từng điểm để cập nhật firmware là không khả thi. Hệ thống hỗ trợ nâng cấp firmware từ xa qua mạng NB-IoT.

- **Giao thức:** HTTP/CoAP qua modem BC660K-GL NB-IoT.
- **Bootloader:** MCUboot (tích hợp sẵn trong Zephyr) quản lý hai phân vùng Flash: Active Slot (đang chạy) và Secondary Slot (chứa firmware mới tải về).
- **Quy trình:** ThingsBoard gửi lệnh update qua MQTT RPC → Firmware tải file .bin về Secondary Slot → Reboot → MCUboot kiểm tra và copy sang Active Slot → Chạy firmware mới.
- **Rollback tự động:** Nếu firmware mới lỗi và crash trong 60 giây đầu, MCUboot tự rollback về firmware cũ. Thiết bị không bị brick.
- **Điều kiện an toàn:** Chỉ nhận lệnh OTA khi đang ở BÌNH THƯỜNG (không đang trong sự kiện lũ) và pin ≥ 30%. Tránh mất điện giữa chừng khi đang ghi flash.

VIII. Tối ưu hóa năng lượng

Mục tiêu: thiết bị phải sống ít nhất 4 tháng không thay pin ở chế độ BÌNH THƯỜNG — ngay cả ở install_type khắt khe nhất (lu_quet, chu kỳ 5 phút). Để đạt được điều này, firmware áp dụng ba nguyên tắc:

1. Tách biệt tầng đo và tầng truyền tải

Ở BÌNH THƯỜNG, tầng đo (MCU thức dậy, đo cảm biến) và tầng truyền tải (bật Modem, gửi data) hoạt động độc lập nhau. MCU đo xong là ngủ lại ngay — không đợi Modem. Modem chỉ bật khi thực sự cần gửi dữ liệu (vào KHẨN CẤP).

- **Kết quả:** Giảm tới 98% điện năng giao tiếp vô tuyến so với cách gửi sau mỗi lần đo.

2. System OFF giữa các chu kỳ đo

Giữa các chu kỳ đo, MCU vào trạng thái **System OFF** — mức tiêu thụ thấp nhất của nRF52840, chỉ ~2 µA. Chỉ RTC vẫn chạy để đánh thức MCU đúng chu kỳ. Modem BC660K tắt hoàn toàn qua tín hiệu enable LDO.

3. PSM và eDRX khi giữ kết nối EMERGENCY

Ở KHẨN CẤP, modem phải giữ kết nối liên tục — không tắt được hoàn toàn. Firmware cấu hình Power Saving Mode (PSM) và extended Discontinuous Reception (eDRX) với mạng NB-IoT: modem chỉ "tỉnh" đúng 30 giây một lần để gửi data, ngủ lại giữa các lần. Giảm dòng idle từ ~3 mA xuống < 0.5 mA.

4. Bảng dòng điện ước tính

Hoạt động	Dòng điện	Thời gian	Ghi chú
MCU System OFF (ngủ)	**~2 µA**	Suốt chu kỳ	Thành phần tiêu thụ thấp nhất
MCU tích cực (đo)	**~5 mA**	~100 ms/chu kỳ	Sensor warmup + đo + tính toán
Cảm biến JSN-SR04T	**~3 mA**	~50 ms/chu kỳ	Tắt GND qua transistor Q5 khi không đo
Modem BC660K tắt	**< 10 µA**	Suốt BÌNH THƯỜNG	Tắt hoàn toàn qua LDO enable
Modem BC660K idle	**~3 mA**	Khi giữ kết nối	EMERGENCY không có PSM/eDRX
Modem BC660K (PSM/eDRX)	**< 0.5 mA**	EMERGENCY có PSM	Giữa các lần gửi 30 giây
Modem BC660K truyền	**~100–300 mA**	~1–3 giây/gói	Peak — cần tụ lọc, kiểm tra sụt áp

Phụ lục A — Cấu trúc JSON payload gửi lên ThingsBoard

Mọi gói tin đều có cùng cấu trúc 10 trường cố định. Firmware không thêm, không bớt trường. Toàn bộ logic cảnh báo đặt tại backend ThingsBoard (Rule Chain) — firmware chỉ gửi dữ liệu đo lường trung thực.

```
// Ví dụ payload đầy đủ — compact, không có whitespace
{
  "water_distance_cm"       : 145,         // int, cm — raw từ JSN-SR04T.
-1 = sensor lỗi
  "water_level_cm"          : 55,          // int, cm — =
install_height_cm - water_distance_cm
  "water_level_pct"         : 34,          // int, %  — CÓ THỂ > 100.
Backend KHÔNG cap.
  "rate_of_rise_cm_per_min" : 0.8,         // float, cm/phút. Dương=dâng,
âm=rút
  "temperature_c"           : 28.5,        // float, °C — nhiệt độ board
mạch, không phải ngoài trời
  "battery_pct"             : 85,          // int, % — tính từ ADC voltage
curve LiPo
  "battery_mv"              : 3850,        // int, mV — điện áp raw ADC
P0.04/AIN2
  "rssi_dbm"                : -87,         // int, dBm — lấy từ AT+QCSQ
sau kết nối
  "report_type"             : "periodic",  // "periodic" (BÌNH THƯỜNG) |
"event" (KHẨN CẤP)
  "ts"                      : 1717459200000 // int, ms — Unix timestamp từ
RTC. Backend dùng cái này!
}
```

Trường	Loại	Edge case quan trọng cần xử lý trong firmware
water_distance_cm	int	-1 khi sensor timeout hoặc I2C lỗi. Không tính vào mực nước. Đếm fault_count.
water_level_cm	int	Có thể âm khi chưa có nước — bình thường, không phải lỗi. -1 khi sensor lỗi.
water_level_pct	int	Cho phép > 100. Không cap. Ví dụ: 125% = nước đã vượt sensor 25%.
rate_of_rise_cm_per_min	float	0.0 ở lần đo đầu tiên (chưa có lần trước để tính delta). Âm khi nước rút.
temperature_c	float	null nếu STTS22H I2C lỗi. Không ảnh hưởng đến alarm ngập. Chỉ dùng giám sát sức khỏe board.
battery_pct	int	Tạo Alarm khi < 15%. Tắt OTA khi < 30%.
rssi_dbm	int	< -105 dBm = vùng yếu, khả năng mất gói cao. Backend tạo Alarm Signal_Weak.
report_type	string	Backend dùng để ưu tiên: "event" xử lý ngay, "periodic" có thể queue.

`ts`	int	ThingsBoard phải dùng trường này làm timestamp — không dùng server timestamp.

Phụ lục B — Timeline sự kiện lũ quét mẫu

Mô phỏng thiết bị `install_type = lu_quet`, `install_height_cm = 200`, lắp tại thượng nguồn suối, cách khu dân cư 30 phút dòng chảy.

Đọc bảng này để hiểu firmware chuyển trạng thái như thế nào trong điều kiện thực tế:

Thời gian	water_level_cm	Pct	rate (cm/ph)	Chế độ	report_type	Ghi chú firmware
+0:00	**25 cm**	16%	`0.0`	**BÌNH THƯỜNG**	`periodic`	Mực nước thấp, trời khô
+0:30	**42 cm**	27%	`0.6`	**BÌNH THƯỜNG**	`periodic`	Bắt đầu mưa thượng nguồn, rate thấp
+1:00	**61 cm**	39%	`1.5`	**BÌNH THƯỜNG**	`periodic`	Nước dâng chậm, chưa đến ngưỡng
+1:10	**68 cm**	44%	`6.2`	**→ KHẨN CẤP**	`event`	rate = 6.2 > 5.0 cm/ph → vào EMERGENCY, bật Modem ngay
+1:11	**80 cm**	52%	`14.0`	**KHẨN CẤP**	`event`	Gửi ngay lên ThingsBoard. Alarm kích hoạt.
+1:15	**115 cm**	74%	`14.0`	**KHẨN CẤP**	`event`	Lũ đang mạnh, gửi mỗi 30 giây
+1:25	**185 cm**	119%	`8.5`	**KHẨN CẤP**	`event`	water_level_pct > 100 — nước vượt vị trí lắp sensor
+1:45	**142 cm**	91%	`-6.2`	**KHẨN CẤP**	`event`	Nước rút, confirm_count vẫn = 0 (chưa đủ 2 chu kỳ)
+1:58	**52 cm**	33%	`-4.1`	**KHẨN CẤP**	`event`	Dưới ngưỡng, confirm_count = 1
+2:00	**48 cm**	31%	`-2.8`	**→ BÌNH THƯỜNG**	`periodic`	confirm_count = 2 → hạ cấp, tắt Modem

> ✓**Cửa sổ cảnh báo còn 20 phút:** Firmware chuyển KHẨN CẤP lúc +1:10 (khi rate vượt 5 cm/phút, mực nước mới 44%). Khu dân cư cách 30 phút dòng chảy — lũ đến lúc +1:40. Còn 30 phút để sơ tán. Đây là ưu thế cốt lõi của việc dùng threshold_rate để phát hiện sớm.